

CS & IT ENGINEERING



Algorithms

Miscellaneous

Lecture No.- 03



By- Aditya sir

Recap of Previous Lecture



Topic

Topic

OBST

↳ Type 1

Topics to be Covered



Topic

Topic

1) OBST

↳ DP

★ 2) Backtracking

ABOUT ME

Hello, I'm Aditya.

1. Represented college as the first Google DSC Ambassador.
2. The only student from the batch to secure an internship at Amazon.
3. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt
- City topper
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10. Joined Dream11 as a Data Scientist.
9. Have mentored around 10k working professionals in field of Data Science and Analytics
10. Have been mentoring GATE aspirants to secure a great rank in limited time.
11. Have got around 27.5K followers on LinkedIn where I share my insights and guide students and professionals.



By- Aditya sir



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : OBST

#Q. How many different BSTs for a given Binary Tree with n nodes?



Topic : OBST



Solution:

- Catalan Number = $\frac{1}{(n+1)} \times {}^{2n}C_n$
- Cost (T) = Cost (successful searches) + Cost (unsuccessful searches)

$$= \sum_{i=1}^n P_i * \text{level}(a_i) + \sum_{i=0}^n q_i * (\text{level}(E_i) - 1)$$



Topic : OBST

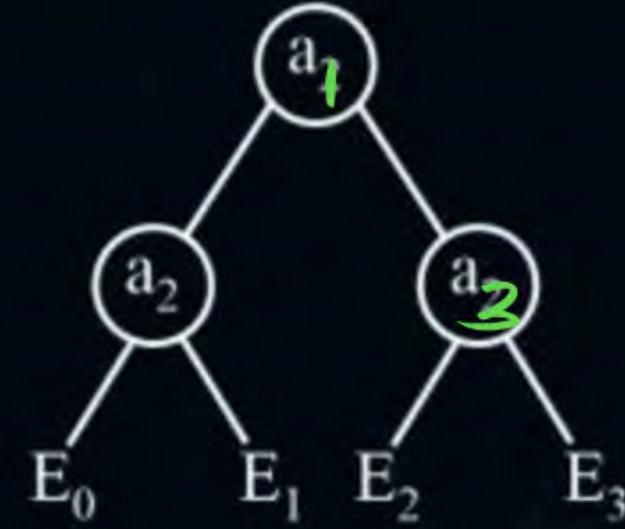
n-identifiers:

$$a_1 \rightarrow a_n$$

$$E_0 \rightarrow E_n$$

Conclusion:

- The cost of BST is dependent on both :
 - the height (depth) as well as
 - the probabilities (p_i & q_i) → frequencies of identifiers





Topic : OBST

Type-2: Construction of Optimal Cost BST (DP based)

Input: $A = [a_1, a_2, \dots, a_n]$

$P = [p_1, p_2, \dots, p_n]$

$Q = [q_0, q_1, \dots, q_n]$

- In a BST with n nodes:
 $n \rightarrow$ identifiers (a_i)
 $(n + 1) \rightarrow$ null links (E_i)

$[E_0 \rightarrow E_n]$



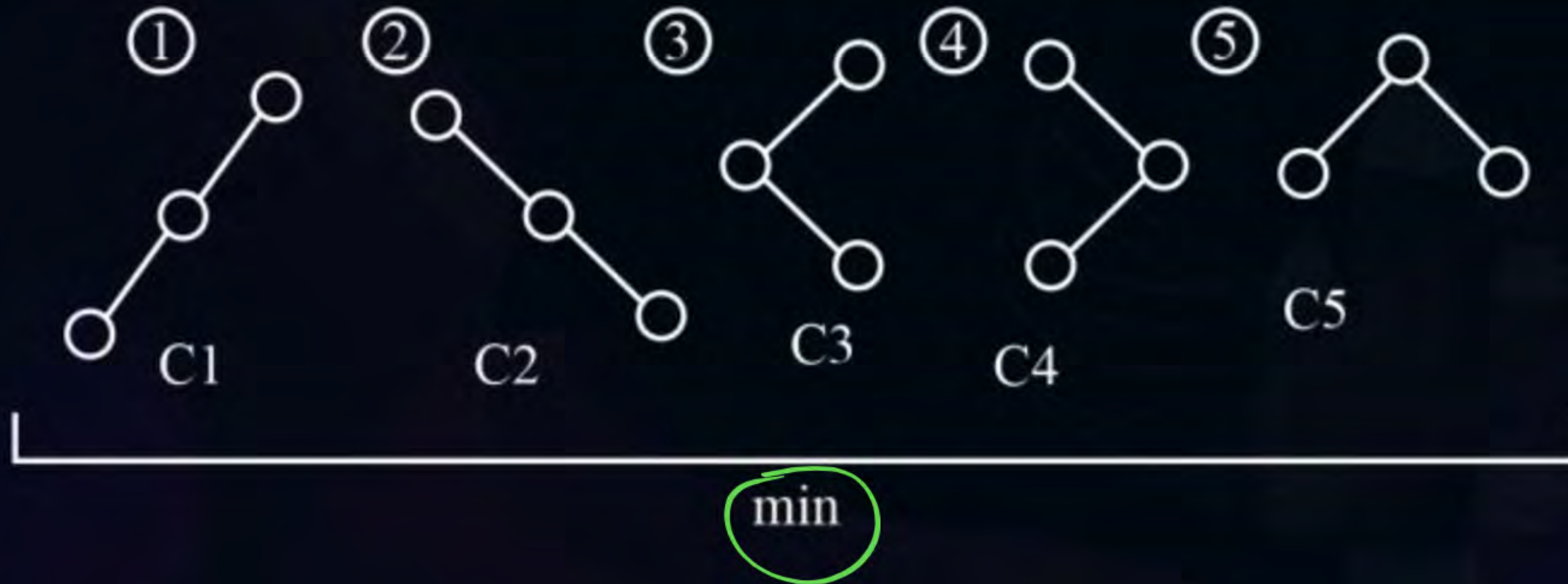
Topic : OBST



E.g.: $n = 3$, no. of different BSTs. = 5

Process of Enumeration / Brute-Force:

$n=4$?





Topic : OBST

No. of nodes	No. of diff BSTs
$n = 3$	5
$n = 4$	14

$n = 4$

$$\begin{aligned} &= \frac{1}{(4 + 1)} \times {}^8C_4 \\ &= \frac{1}{5} \times \frac{8!}{4! \times 4!} \\ &= \frac{1}{5} \times \frac{8 \times 7 \times 6 \times 5 \times 4!}{4! \times 4!} \\ &= \frac{8 \times 7 \times 6}{4 \times 3 \times 2} \\ &= 2 \times 7 = 14 \end{aligned}$$



Topic : OBST



Imagine : $n = 300$

$$\text{No. of BSTs} = \frac{1}{(301)} \times {}^{600}C_{300}$$

- Hence, Brute Force Approach is **NOT** feasible for large n . (As no. of BSTs increase exponentially with increase in n)
- ~~Some~~ ^{we} need a better approach/optimal.
- Dynamic Programming (DP)





Topic : OBST



DP based Approach for Construction of OBST:

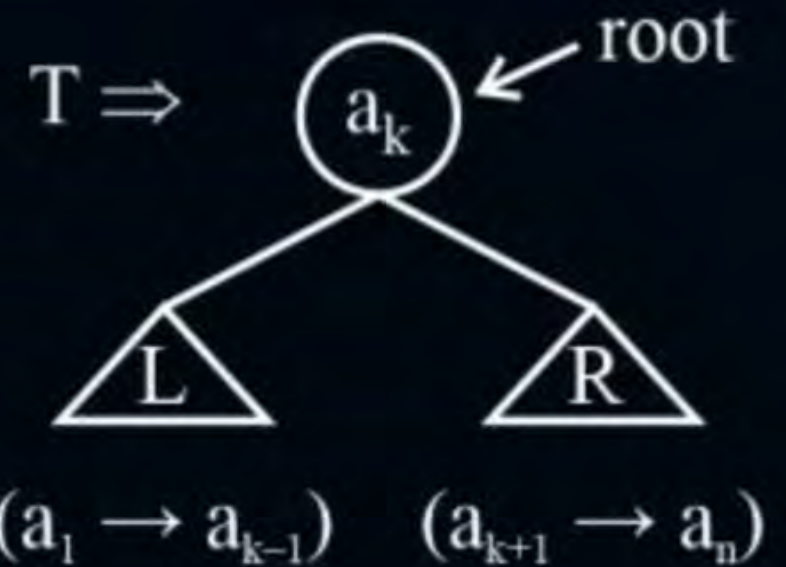
$$A = [a_1, a_2, \dots, a_n]$$

$$\text{Cost}(T) = \text{Cost}(a_k) + \text{Cost}(L) + \text{Cost}(R)$$

$a_k \rightarrow$ root node

For root, level = 1

$$\text{Probability} = \underline{P_k}$$



$$\text{Cost}(a_k) = \text{Prob}(a_k) \times \text{level}(a_k)$$

$$= \underline{P_k} \times \underline{1}$$
$$= \underline{P_k}$$



Topic : OBST

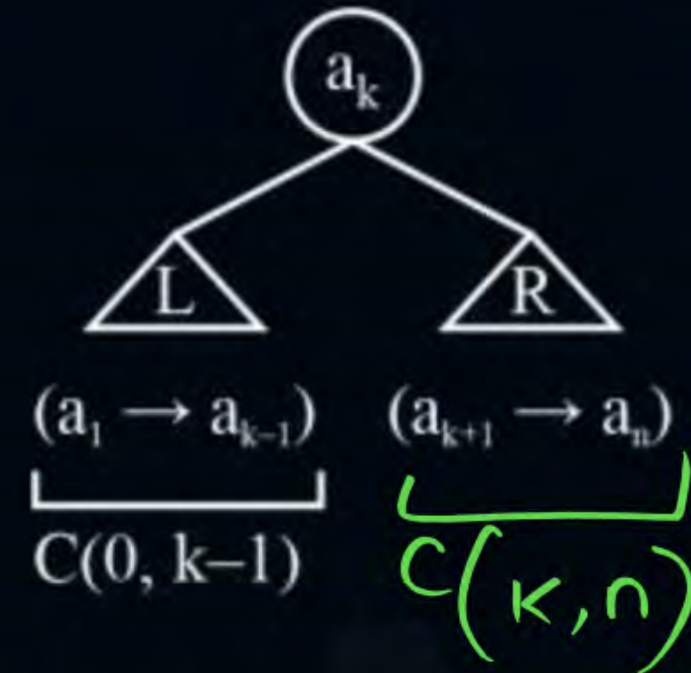
$$\text{Cost}(T) = \text{Cost}(a_k) + \text{Cost}(L) + \text{Cost}(R)$$

$$\text{Cost}(T) = P_k + \text{Cost}(L) + \text{Cost}(R)$$

Let $C(0, n)$ represent cost of BST $T_{1 \rightarrow n}$

$$\text{Cost}(T_{1 \rightarrow n}) = C(0, n) = \{P_k + \underbrace{C(0, k-1)} + \underbrace{C(k, n)}\} + \underbrace{W(0, n)}; \quad 0 < K \leq n$$

★



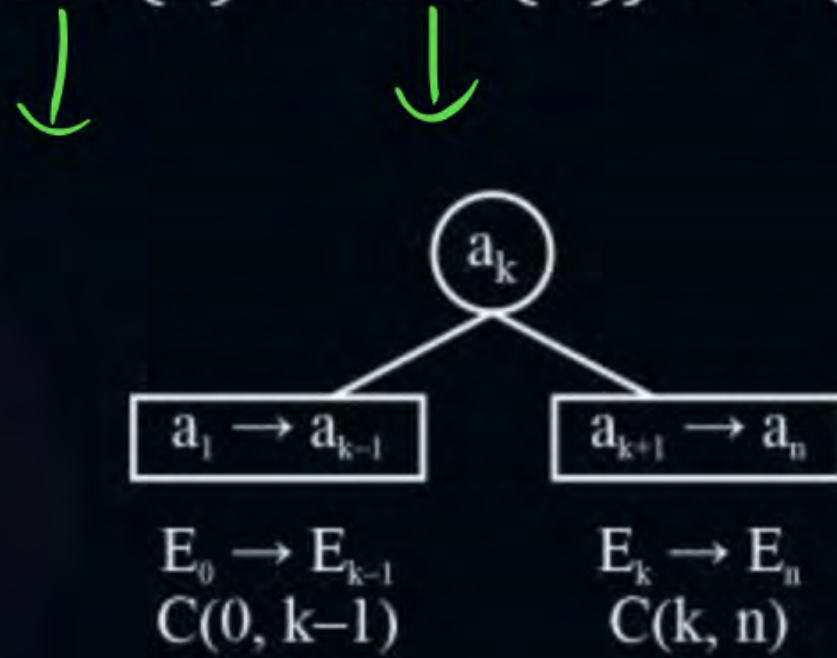
NOTE: $W(0, n)$ is added to the cost of LST and RST to balance it with respect to the root.



Topic : OBST

Explanation:

- $$T_{1 \rightarrow n} = C(0, n) = \{P_k + \text{Cost (L)} + \text{Cost (R)}\} + W(0, n)$$





Topic : OBST



(Bonus)

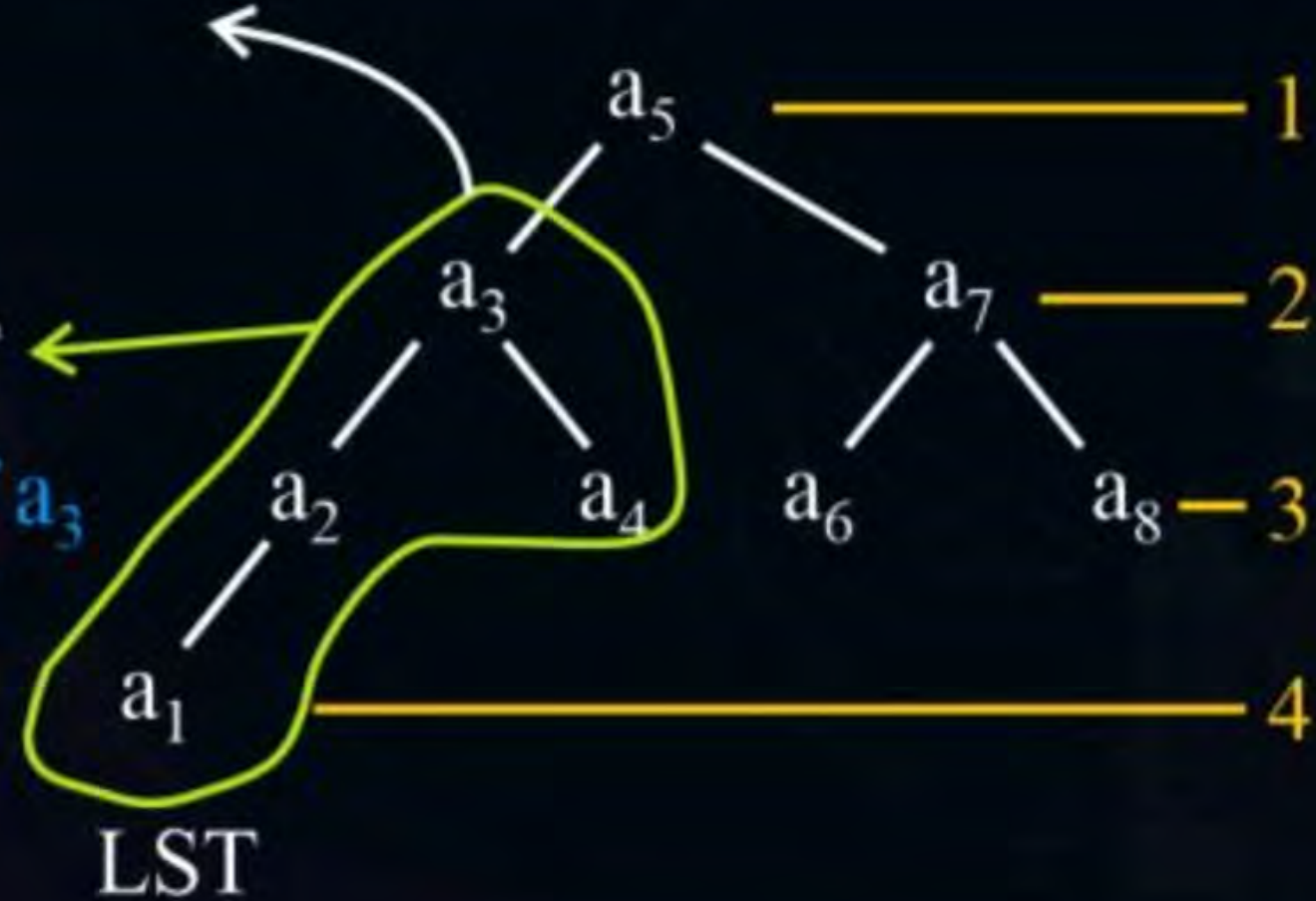
Need for $W(0, n)$ → Explanation (~~intention~~):

Overall level of a_3 ?

⇒ 2

a_2 is
root of LST

→ Level of a_3
within LST
is 1





Topic : OBST

Generalizing for any subtree a_{ij} (root is a_k):

$$C(i, j) = \min \{P_k + C(i, k-1) + C(k, j) + W(i, j)\}; i < k \leq j \quad \dots(i)$$

where k is such that, having a_k as root will minimize equation (i)

Base Condition, $C(i, j) = 0$ (0 nodes)



Topic : OBST

NOTE:

Time Complexity of Previous DP based approach.

$$\boxed{TC = O(n^3)} \quad \{DP\}$$

Brute Force → DP based



Expo



$O(n^3)$ Polynomial

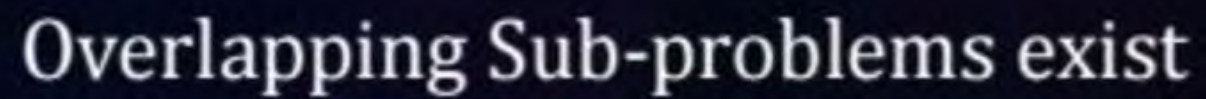


Topic : OBST

Check for overlapping Sub-problems:



E.g.: $A = [a_1, a_2, a_3, a_4]$


$$i < k \leq j$$




Topic : Backtracking and Branch-Bound





Topic : Back Tracking

- Represents one of the most general techniques.
- Many problems which deal with searching for a set of solutions or which ask for an optimal solution satisfying some constraints can be solved using the backtracking formulation.
- In order to apply the backtrack method, the desired solution must be expressible as an n -tuple $(x_1, x_2, \dots, x_n)$ where the x_i are chosen from some finite set S_i .
Bonus Often the problem to be solved calls for finding one vector which maximizes (or minimizes or satisfies) a criterion function $P(x_1, \dots, x_n)$. Sometimes it seeks all such vectors which satisfy P .
- For example



Topic : Back Tracking

E.g.: n-Queens Problem:

- Given n Queens ($q_1, q_2, \dots, q_n$)

$X = [x_1 \rightarrow x_n] \rightarrow$ feasible solution

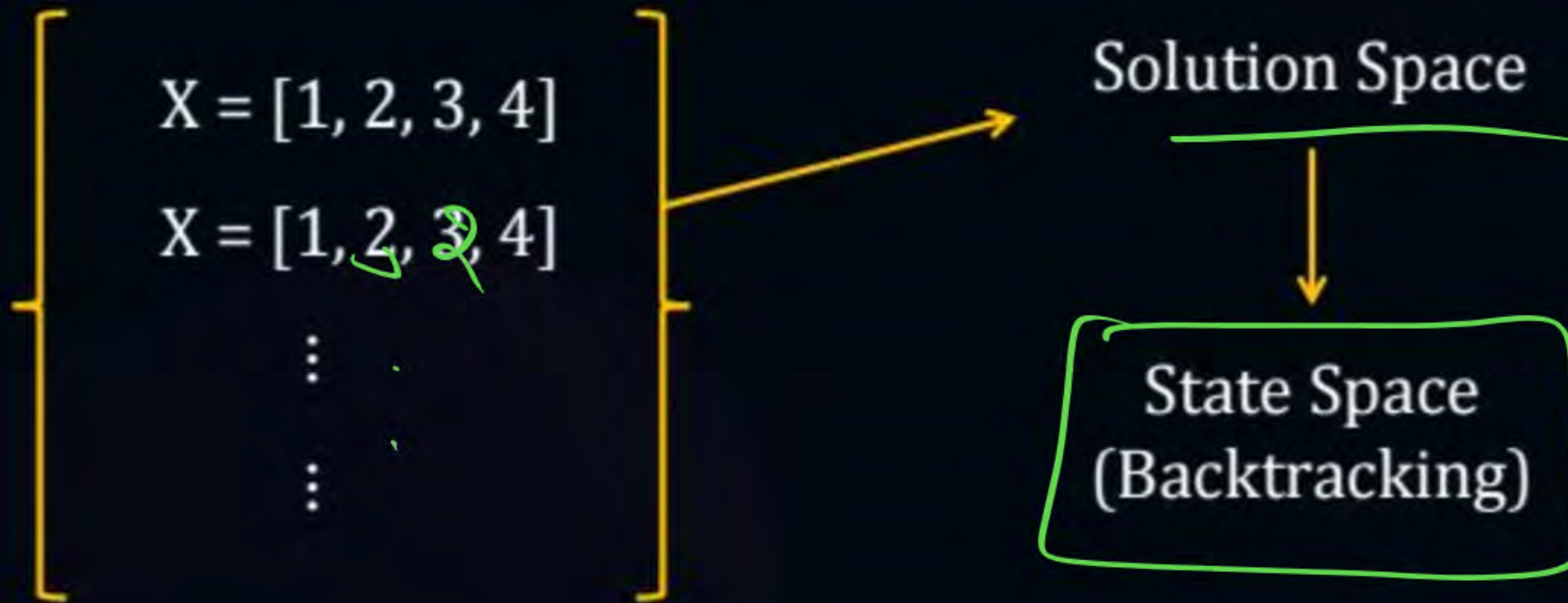
$X = [x_1, x_2, \dots, x_n]$

X_i = Position of Queen Q_i in the i^{th} row of $n \times n$ matrix (chess board)

$$1 \leq X_i \leq n$$



Topic : Back Tracking





Topic : Back Tracking

E.g.: $n = 4$ (q_1, q_2, q_3, q_4)

$X = [1, 3, _, _]$

$X = [1, _, _, _]$

$X = [2, 4, 1, 3] \rightarrow$ feasible solution

$1, 3, _, _$

	1	1	1	1
q_1	$\times$	$\bullet$		
q_2	$\times$	$\times$	$\times$	$\bullet$
q_3	$\bullet$			
q_4	$\times$	$\times$	$\bullet$	



Topic : Back Tracking



State Space Tree:

(1) Fixed Size Tuple:

E.g.: n-Queens

(2) Variable Size Tuple:

E.g.: Sum of Subsets (SOS)



Topic : Back Tracking

State Space Tree of Brute Force :





Topic : Back Tracking



$X = [1, 2, 3, 4]$

$X = [1, 2, 3, 4]$

$\vdots$

$\vdots$

Total branches = $6 \times 4 = 24$

$n!$ = Size of solution space

$n = 4$

$4! = 24$



Topic : Back Tracking



Important Points:

- (1) Backtracking uses DFS.
- (2) DFS is used to find the feasible solution among all the solutions in the state space.
- (3) It applies a bounding function to terminate/stop those nodes that can never lead to an optimal/feasible solution.



Topic : Back Tracking



Backtracking:

- ~~DGS~~ along with some Bounding Function.

DFS



Topic : Back Tracking



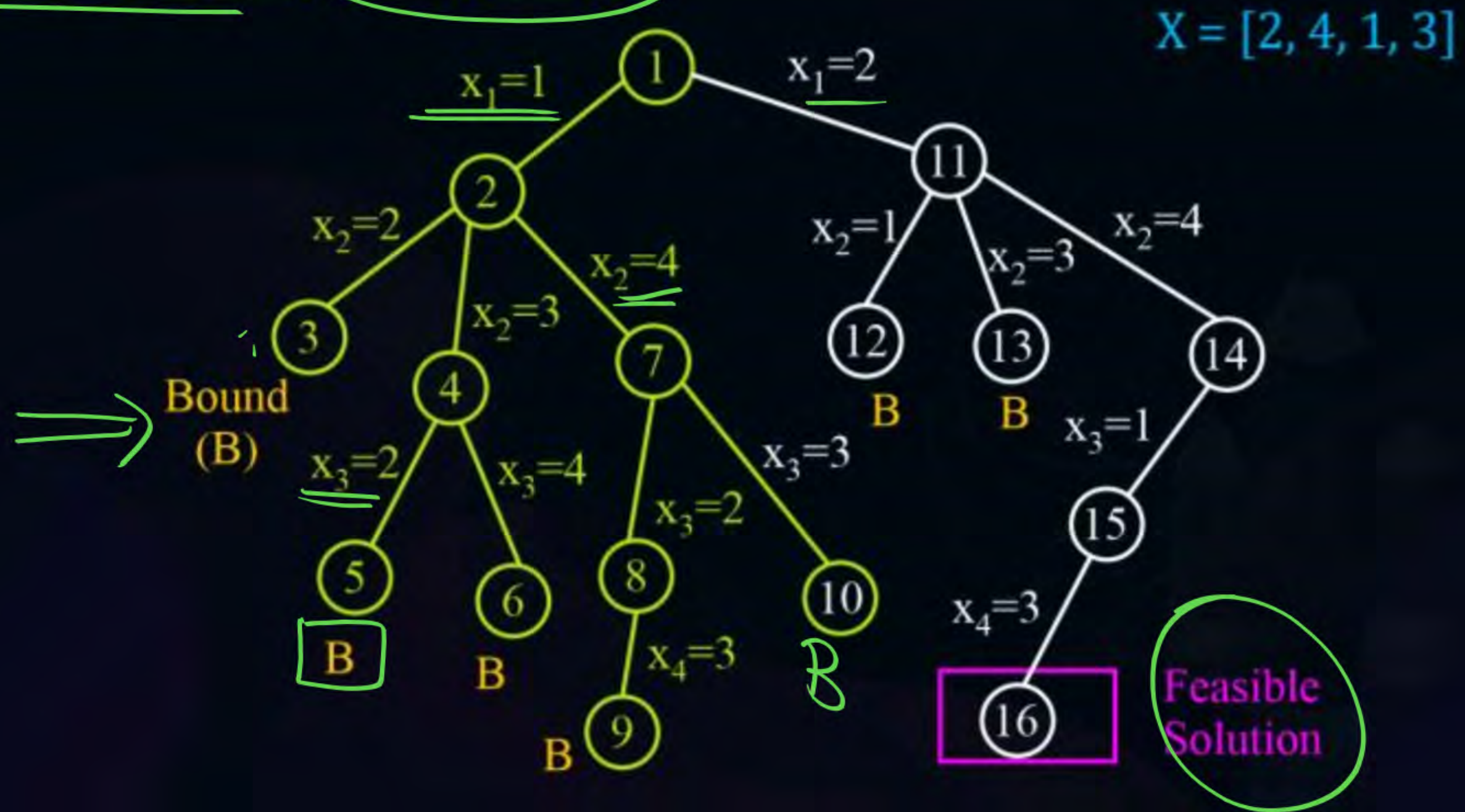
Potential Question:

- Portion of the State Space Tree that gets generated due to backtracking process for $n = 4$ Queens Problem.
- To get 1st feasible solution: 16 states



Topic : Back Tracking

State Space Tree using Back-tracking:





Topic : Back Tracking

	1	1	1	1
q_1		✓		
q_2				✓
q_3	✓			
q_4		✓	✓	





Topic : Back Tracking

Example of Variable Sized Tuple Problem and Solution using Backtracking

(State Space Tree):

Sum of Subset (SOS):

- $A[n]$: n elements $A_1, \dots, A_n$; $M = \text{Target sum}$



Topic : Back Tracking

E.g.:

$n = 6, M = 50$ (Target Sum)

$A = [20, 40, 50, 10, 30, 5]$

indices

$S1 = \{3\}$

$S2 = \{2, 4\}$

$S3 = \{1, 5\}$

feasible solution

Variable sized tuple as the feasible solutions are not at the same level (size) in the state space Tree.



Topic : Back Tracking

Sum of the State Space for SOS problem with 'n' elements:

= Number of Subsets

$$= 2^n$$

1, 2, 3, ..., n

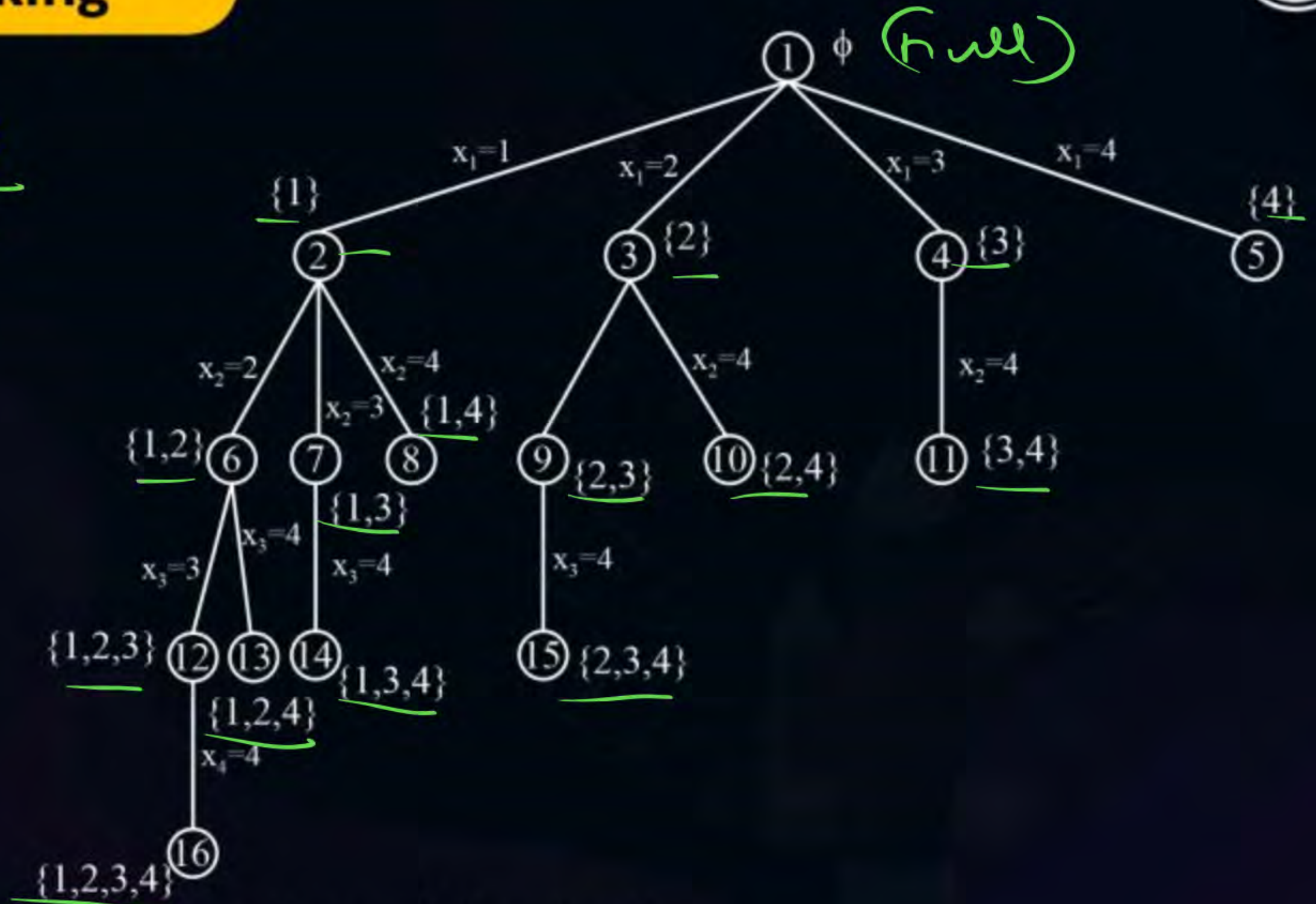
✓ ↙ ↘

$$2 \times 2 \times 2 \times \dots \times 2$$
$$(2^n)$$



Topic : Back Tracking

State Space Tree for $n = 4$



In this tree, each node is representing 1 possible subset.



Topic : Back Tracking



$\{a_1, a_2, a_3, a_4\}$

$\rightarrow ?$

$$n = 4$$

$$2^n = \underline{\underline{16}}$$

ϕ

a_1

a_2

a_3

$a_1 a_2$

$a_1 a_3$

$a_2 a_3$

$a_1 a_2 a_3$

$$n = 3$$

$$2^3 = \textcircled{8}$$

$$\text{Subsets} = \underline{\underline{2^3 = 8}}$$



Topic : Back Tracking



For 4 elements, how many subsets?

$$= 2^4 = 16$$

And no. of nodes/states in Tree also 16.



Topic : Back Tracking



Test: Graph Coloring

- You are given a graph having n vertices/nodes.
- You have to color this graph using minimum colors such that no two adjacent nodes have the same color.





Topic : Back Tracking



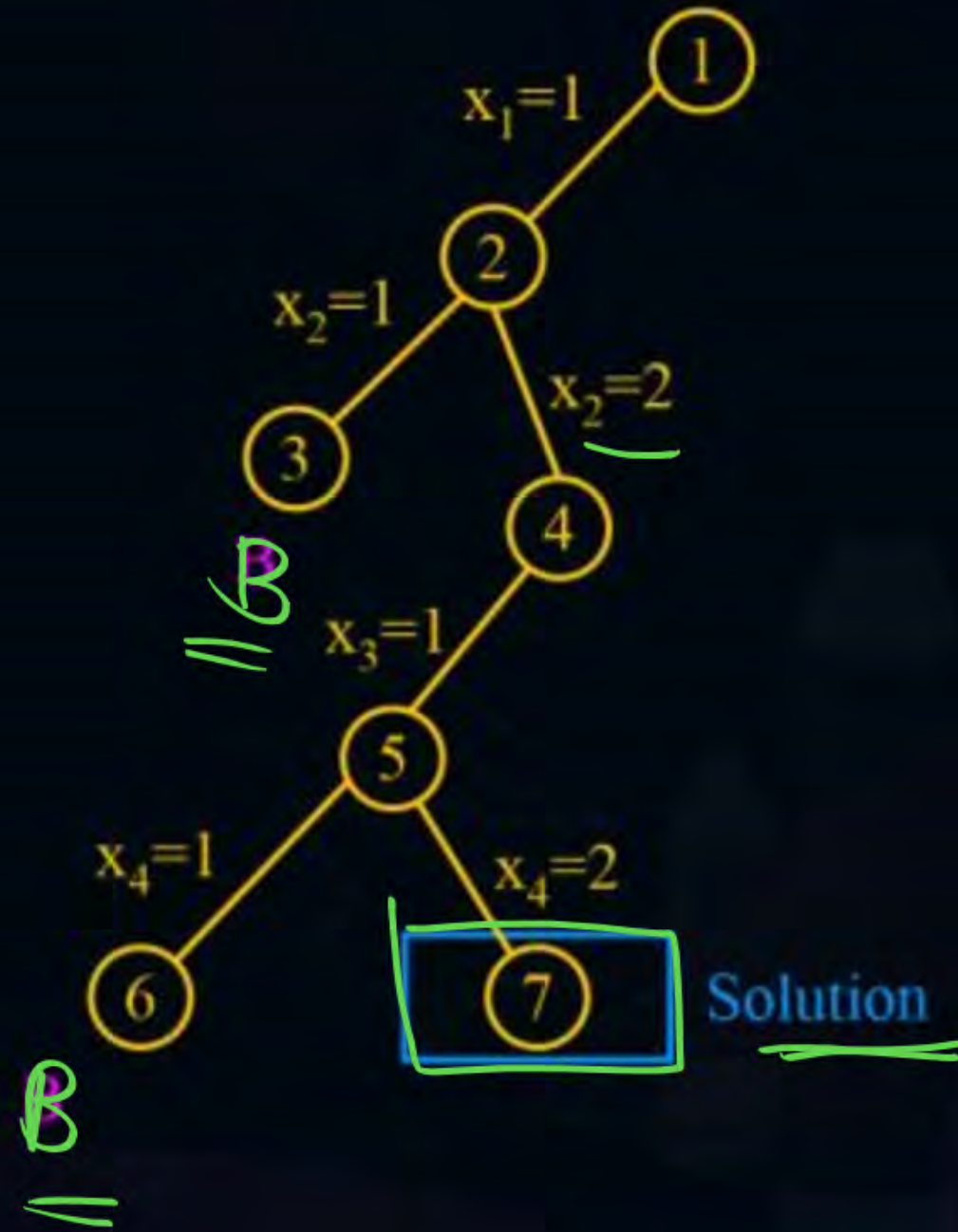
Optimal

#Q. How many States gets generated for 1st ~~optional~~ solution.





Topic : Back Tracking





Topic : Back Tracking

Branch and Bound

HW

- Mostly same idea as that of Backtracking but it uses different Searching approach.
- Backtracking $\rightarrow$ DFS + Bounding Function
- Branch & Bound $\rightarrow$ BFS + Bounding Function

DFS $\rightarrow$ Stack

BFS $\rightarrow$ Queue



Summary



OBST $\rightarrow$ DP

Backtracking



THANK - YOU